

Forge & Flow

Architecture Guide

The plain-English and technical reference for how Forge & Flow is built - every decision, every layer, every reason. Study notes for the founder.

Table of Contents

01	What Forge & Flow Is - Plain English
02	Glossary - Every Term, Every Acronym
03	The Eight Hard Architectural Promises
04	The Tech Stack - Every Piece, What It Does, Why
05	Architecture Overview - How the Pieces Fit Together
06	Data Flow - How Information Moves Through the System
07	Logic Layers - Where Code Lives and Why
08	Schema Decisions - The Database Lock-Ins
09	AI Infrastructure - The Heart of the Product
10	Security and Privacy
11	The Build Phase Plan - Sequential Cadence
12	Decision Log - Q1 through Q12 with Rationale
13	Future Flags - Items Parked for Later Decision
14	Quick Reference Card

What Forge & Flow Is

Plain English

Forge & Flow is a phone and tablet application for restaurant managers. It connects to the systems a restaurant already uses - the point-of-sale system, the labor scheduling system, the reservation system - and makes sense of what the data is telling the operator. It does two things at once: it shows the operator what is happening in their restaurant in clear summaries, and it gives them an artificial-intelligence assistant called the Advisor that can answer questions about their operation in plain language.

The big idea behind the Advisor is that it does not give generic restaurant advice. It understands the specific restaurant's patterns - which servers are scheduled, what the weather has been, which menu items have been on the specials board, how labor cost ratios have moved week over week - and it grounds its recommendations in those actual facts plus the operator's chosen methodology. When the Advisor recommends something, it shows its reasoning ("I recommend cutting four labor hours on Wednesday because shifts on rainy Wednesdays in this category historically run twelve percent below forecast"). The operator decides whether to act. The product never takes action on the operator's behalf at launch.

Behind the Advisor is a body of operational methodology - an editable knowledge base that the founder, Vanessa, maintains. New methodology documents drop into the knowledge base from a Settings page in the application; the application chunks them, fingerprints each chunk by content, computes vector embeddings via a service called Voyage, and writes them into a database. When an operator asks a question, the Advisor retrieves the most relevant chunks plus the operator's actual data and asks Anthropic's Claude language model to reason over them.

The product runs in a paid subscription model where Forge & Flow holds all the application-programming-interface keys for the artificial-intelligence services and absorbs that cost as part of doing business. Restaurants never manage keys. The keys live on a server-side proxy that brokers every call.

Technical Summary

Forge & Flow is a Flutter mobile application written in the Dart programming language, with a server-side proxy backend hosted on Google Cloud Run, a Postgres database hosted on Supabase with the Apache AGE graph extension and the pgvector vector-search extension,

and a knowledge-corpus pipeline that uses Voyage embeddings (model `voyage-4-large`, 1024 dimensions) and Voyage rerank (model `rerank-2.5`) to feed grounded context to Anthropic's Claude language models (Haiku for routine questions, Sonnet for nuanced ones).

The application stores operational data locally in SQLite for offline-first operation, and synchronizes to Postgres on the server. Every operator-scoped fact in Postgres carries an `(operator_id, location_id)` pair plus row-level-security policies, so multi-tenant isolation is database-enforced. Time-sensitive data is stored as Coordinated Universal Time (UTC) timestamps with a denormalized `business_date` column computed at write time using each location's time zone. Application code never queries operator-scoped tables directly - it goes through a repository wrapper that injects `(operator_id, location_id)` from the active session context. Postgres row-level-security is the safety net underneath that wrapper, not the primary defense.

The Advisor is a single-shot Retrieval-Augmented Generation system at launch (one question, one Claude call, one answer). Phase 12 post-launch will add agentic workflow tools that can chain tool calls, with hard bounds on tool-call count, token spend, and wall-clock time, plus mandatory human approval for any irreversible action.

NORTH STAR

Point-of-sale & Labor & Reservation systems → Canonical operational facts → 60-Day Benchmark Snapshot → TargetCycle & DemandForecastContext → SchedulePlan → WeeklyPlanSnapshot → Shift → Variance → History → Learn. Each arrow is a clearly-owned service in the codebase. Nothing skips a step.

Glossary - Every Term, Every Acronym

Read this first if anything below feels unfamiliar. No abbreviation appears anywhere in this document without an entry here.

Foundation Terms

API (Application Programming Interface) - The contract a software service exposes so other software can talk to it. The Anthropic API is the network endpoint Forge & Flow's proxy backend calls to ask Claude to generate an answer. Restaurants never see this directly.

Application - The Flutter mobile/tablet app that operators use. Lives in `lib/` in the source repository. Two flavors: Forge & Flow (manager-facing, `lib/main_forgeflow.dart`) and Barrio (staff-facing, `lib/main_barrio.dart`).

Backend / Proxy Backend - A small server-side service hosted on Google Cloud Run. It holds all the production API keys, brokers every call to Anthropic and Voyage, enforces per-operator cost caps and rate limits, and is the only place production keys ever live. The Flutter app never holds production API keys.

BYO-key (Bring Your Own Key) - A pricing pattern where customers supply their own API keys for paid services. Forge & Flow does **not** use this pattern. Restaurants do not manage keys; Forge & Flow holds all keys server-side and absorbs cost as a subscription expense (Hard Promise #7).

Codex / Claude Code - The AI assistant the founder uses to write code. Codex generates implementation prompts; Claude (a separate instance) carries them out in the codebase. The two roles are the human's collaborators, not part of the product itself.

Database & Storage Terms

Apache AGE (A Graph Extension) - An open-source extension to Postgres that lets the Postgres database act like a graph database in addition to a relational table database. Used by Forge & Flow so the Advisor can perform graph traversals over the knowledge graph (concepts connected by edges) without running a second separate database service. The graph extension and the vector extension and the relational tables all coexist in one Postgres instance.

Business Date - The restaurant's operating date, which is not always the same as the calendar midnight date. A restaurant may consider a sale rung at 1:30 in the morning to belong to the prior business date. Business date is the master operational anchor in Forge & Flow's time model. See Section 8.

Cypher - The query language used to write graph queries against Apache AGE. Conceptually similar to Structured Query Language (SQL) but built for graph traversal: `MATCH (a) -[:CALLS]->(b) RETURN b` instead of joining tables.

DST (Daylight Saving Time) - The seasonal clock shift in many regions. Spring-forward removes one hour; fall-back duplicates one hour. Time-zone-naive timestamps cannot survive DST without silent corruption - the 2:30 AM hour on fall-back appears twice with no way to distinguish them. Forge & Flow bans timezone-naive timestamps in operator-scoped tables for exactly this reason.

Embedding - A list of numbers (a vector) that represents the meaning of a piece of text. Two pieces of text that mean similar things produce similar vectors. The Advisor uses embeddings to find which methodology chunks are relevant to an operator's question - it computes the embedding of the question, then looks up chunks whose embedding vectors are nearest. Voyage's `voyage-4-large` model produces 1024-dimensional vectors.

IANA Time Zone (Internet Assigned Numbers Authority) - The global registry of named time zones - strings like `America/Toronto` or `Europe/Madrid`. Forge & Flow stores each location's time zone as an IANA string so the system always knows how to convert a UTC timestamp to that location's local clock, including DST.

pgvector - A Postgres extension that adds a vector data type and similarity-search functions. Forge & Flow uses pgvector to store the 1024-dimensional Voyage embeddings of corpus chunks and to find the chunks most similar to a query.

Postgres / PostgreSQL - The open-source relational database Forge & Flow uses on the server side. Hosted on Supabase. Extended with Apache AGE (graph) and pgvector (vector search) on the same instance.

RLS (Row-Level Security) - A Postgres feature that automatically rewrites every query to only return rows belonging to the currently authenticated operator. Forge & Flow uses RLS as the safety net beneath the application's repository pattern. Both layers must agree before any operator's data is returned. RLS turns on in Phase 9.

SQLite - A small, file-based database that runs inside the Flutter application on the device. Forge & Flow uses SQLite for offline-first local storage. The application reads and writes SQLite locally; data synchronizes to Postgres on the server when the device is online.

Supabase - A managed Postgres service. Forge & Flow rents Supabase to host the production database. During development a local Supabase Docker container provides the same environment.

TIMESTAMPTZ - A Postgres data type that stores a moment in time as Coordinated Universal Time (UTC) with explicit time-zone awareness. The opposite is **TIMESTAMP WITHOUT TIME ZONE**, which is timezone-naïve and prone to silent DST corruption. Forge & Flow's storage rule (Q4) bans **TIMESTAMP WITHOUT TIME ZONE** in operator-scoped tables.

AI & Advisor Terms

Advisor - The AI-powered chat surface inside Forge & Flow. Operators ask questions in plain language; the Advisor retrieves grounded context (corpus + their data) and uses Claude to produce a recommendation. Recommendation-only at launch - never takes action on the operator's behalf.

Anthropic / Claude - Anthropic is the company that builds Claude, the language model Forge & Flow uses for advisor answers. Specific models: **claude-haiku-4-5** (fast, cheap) for routine questions and **claude-sonnet-4-6** (slower, smarter) for nuanced ones. Tier routing decides which.

Capability Flag - A boolean (true/false) tag on a Provider abstraction indicating whether that provider supports a feature: **promptCaching**, **streaming**, **toolUse**, **vision**. Lets generic Forge & Flow code take advantage of provider-specific features without binding to a specific provider.

Content Hash - A fingerprint computed from the bytes of content. Two pieces of content with identical bytes produce identical hashes; any change produces a new hash. Forge & Flow's corpus chunks use content hashes as identifiers (Q6) so old advisor citations remain accurate after methodology updates.

Corpus - The body of methodology documents the Advisor reasons over. Markdown files authored by Vanessa or other founder-level admins, chunked into smaller units, content-hashed, embedded by Voyage, stored in Postgres.

LLM (Large Language Model) - A neural-network model trained on enormous text corpora to generate text in response to prompts. Anthropic's Claude is an LLM. So is OpenAI's GPT and Google's Gemini. Forge & Flow's `LLMProvider` abstraction lets multiple LLM providers plug in behind a common interface.

MCP (Model Context Protocol) - A standard published by Anthropic for letting AI agents call tools (functions). Forge & Flow's MCP tool layer exposes safe, read-only operator data tools to the Advisor at launch. Phase 12 will extend this with write-tools for workflow automation.

Prompt Caching - An Anthropic-specific feature that lets a long system prompt be cached so repeated calls with the same prompt get a roughly tenfold cost reduction on the cached portion. Exposed through Forge & Flow's `Capability.promptCaching` flag.

Prompt Injection - An attack class where untrusted text (such as operator-typed shift notes) contains instructions designed to manipulate the language model. Forge & Flow defends with base64 encoding of operator content, system-prompt scoping, output sanitization for links and images, and length caps. See Section 9.

RAG (Retrieval-Augmented Generation) - A pattern where a language model is given relevant documents to ground its answer, instead of relying purely on its training data. The Advisor is RAG-based: retrieve corpus chunks plus operator data, then ask Claude to reason over them.

Rerank - A second-stage scoring step after vector search. Vector search returns the top-K candidates by embedding similarity (fast, approximate). The reranker then re-scores those candidates using a more expensive but more accurate model. Forge & Flow uses Voyage's `rerank-2.5`.

Streaming - A response mode where an LLM emits tokens one chunk at a time as they are generated, instead of returning the complete answer at once. Forge & Flow ships non-streaming at launch and adds an additive `completeStream()` method later (Q8).

Voyage - The company that builds the embedding and rerank models Forge & Flow uses. `voyage-4-large` for 1024-dim embeddings; `rerank-2.5` for reranking.

Operational & Domain Terms

ActiveTargetProfile - The runtime projection of the currently-active TargetCycle. The system reads from this when computing variance for the current week.

Daypart - A subdivision of a service period - for example, "early lunch" inside the lunch service period. Phase 10.5 introduces daypart views alongside the whole-day Shift view.

DemandForecastContext - Rolling demand information used for upcoming-week planning. Distinct from TargetCycle, which locks 60-day standards.

kDemoMode - A boolean flag in the Flutter codebase that switches the data writer. When true, demo mode uses `mock_integration_replay_seed.dart` to fill SQLite with simulated data. When false, vendor connectors fill the same tables from real point-of-sale and labor systems. Same tables, same read paths, same advisor - just a different writer. Persists post-launch as a real product feature.

LaborModel - The pure-formula service that owns labor-cost calculations. Lives in `lib/domain/services/`. No input/output side effects, just math.

Operator - The restaurant business that subscribes to Forge & Flow. One operator may run multiple physical Locations. Each operator is fully isolated from every other operator at the database level (RLS) and the application level (repository pattern).

SchedulePlan - The week's planned shifts derived from TargetCycle plus DemandForecastContext.

Shift - A single work-shift record. The whole-day view is the authoritative record; daypart views (Phase 10.5) sit alongside without replacing it.

TargetCycle - The 60-day standard window that locks expected operational metrics. Used as the baseline against which actual performance is measured.

Variance - The difference between actual performance and the standard set by ActiveTargetProfile or WeeklyPlanSnapshot.

WeeklyPlanSnapshot - The locked plan for a given week, captured at week-start. Becomes the comparison plan for that week's variance once the week closes.

The Eight Hard Architectural Promises

Every implementation slice respects these. They are written in `CLAUDE.md` at the top of the source tree and in the active phase plan at `docs/phases/post_11a7_stabilization_plan.md`. They are non-negotiable: any work that conflicts with them gets stopped and re-scoped to comply.

1 Phase 8 is a pure transport swap

Plain English. When Forge & Flow goes from running on demo data to running on real point-of-sale and labor data, the only thing that should change is where the data comes from. Same tables, same read paths, same user interface, same advisor.

Why it matters. If Phase 8 turns into a redesign, the team is doing the wrong work. Redesign-flavored discoveries get reassigned to Phase 7.57 (structural cleanup), Phase 7.58 (behavioral alignment), or Phase 7.61 (freshness audit) - never absorbed into Phase 8.

2 Demo mode persists post-launch

Plain English. The demo mode is not a temporary scaffolding. It is a real, supported product feature that ships at launch and lives forever. Investors and prospective operators see the demo. Vanessa runs the demo to test new methodology. Engineers run the demo to develop without touching real customer data.

Technical. A flag named `kDemoMode` in the Flutter codebase chooses the writer. `true` fills SQLite from `mock_integration_replay_seed.dart`. `false` fills the same SQLite tables from vendor connectors (Toast, 7shifts, etc., as those land in Phase 8). Read paths, user interface, and advisor are identical in both modes.

3

No application logic changes before Phase 7.58

Plain English. Phases 7.57 through 9.5 are not allowed to redecide what the application does. They can only move files around, add abstraction layers, write documentation, or add additive new capability. Phase 7.58 is the first slice where someone can argue "this logic should change."

Why. The application's behavior must stay verifiable through the whole stabilization sweep. A behavioral change inside a structural slice corrupts the test that proves the slice is safe.

4

Per-operator isolation is non-negotiable, and the schema is multi-location-ready from day one

Plain English. Operator A must never see Operator B's shifts, schedules, sales, or anything else. And one operator may run five restaurants - the schema has to handle that without a data migration.

Technical. Every operator-scoped fact table (shifts, cycles, plans, weekly snapshots, variance, history) carries `(operator_id, location_id)` from creation. Single-location operators run with a default `location_id`. Corpus and methodology stay `operator_id`-scoped. Phase 9 turns on row-level-security enforcement. Retrofit is not an option once thousands of facts exist.

5

Apache AGE graph projection ships before Phase 11b starts

Plain English. The graph traversal - the part that makes the Advisor smarter than a generic chat model that just looks up documents - has to be real before the Advisor user interface ships. It is the launch differentiator, not marketing.

Technical. Phase 7.57.4 is the slice that lands AGE projection. The retrieval layer in Phase 11a.8 / 11a.9 ships with a fallback: if AGE benchmarks fail at production scale, a flag flips the retrieval mode to vector-only and the Advisor degrades gracefully (Q12).

6

The Advisor speaks in recommendations, not commands

Plain English. The Advisor never takes action on the operator's behalf at launch. It says "I recommend X because Y." The operator decides whether to act. Operator decisions and outcomes are the operator's sole responsibility, codified in Phase 9.8 terms-and-conditions work.

Why. A recommendation-only posture is the legal shield. Liability for restaurant operations stays with the restaurant operator. Phase 12 workflow automation will extend this with explicit per-action operator consent - same posture, just covering automated actions instead of free-text answers.

7

Forge & Flow holds all provider keys server-side

Plain English. Restaurants do not bring their own API keys. Forge & Flow holds the master keys for Anthropic and Voyage, eats the cost as part of the subscription, and brokers every call through a server-side proxy. Restaurants never see, manage, or worry about keys.

Technical. Phase 11a.10 is the proxy backend, split into 11a.10a (infrastructure on Cloud Run with key vault and JWT authentication), 11a.10b (per-operator token budget, rate limiting, monthly cost cap with refusal policy), 11a.10c (admin dashboards, deferred to post-100-locations), 11a.10d (provider fallback chains, capability locked but off by default). Production keys live in Cloud Run environment variables and key-management service. The Flutter application never holds production keys; the dev `--dart-define` path is for local development only.

8

AI infrastructure is general-purpose

Plain English. The artificial-intelligence plumbing was not built for the Advisor specifically. Today the Advisor is one consumer; tomorrow's workflow automation is another. The abstractions are named for capability, not product surface.

Technical. The interfaces are `LLMProvider`, `EmbeddingProvider`, `RerankProvider`, and `DataSourceProvider` - capability-named. The advisor in Phase 11b is one consumer; workflow automation in Phase 12 post-launch will be another. Renaming any advisor-specific naming to general-purpose lands in slice 7.57.3c or 7.57.3d.

The Tech Stack - Every Piece, What It Does, Why

Every external technology Forge & Flow depends on, with the role it plays and the reason it was chosen.

Client Application

Flutter CLIENT FRAMEWORK

Google's cross-platform application framework. One codebase compiles to iOS, Android, web, macOS, Windows, and Linux. Forge & Flow primarily targets iOS and Android tablets that operators use in the back-of-house.

Chosen because: one codebase saves cost; the widget model and Google Fonts integration produce a polished interface; the Dart language is statically typed and easy to reason about; the existing team has Flutter expertise.

Dart PROGRAMMING LANGUAGE

The programming language Flutter applications are written in. Statically typed, garbage-collected, compiles to native code or to JavaScript.

Comes with Flutter; the type system catches whole classes of errors at compile time; ergonomic for asynchronous code (`async` / `await` built in).

SQLite LOCAL DATABASE

A small, file-based database that runs inside the Flutter application on the device. The application reads and writes operational data to SQLite locally; data synchronizes to Postgres on the server when the device is online.

Offline-first operation is required - restaurants in service must keep working when network connectivity drops. SQLite is the proven local store for mobile apps.

Google Fonts: Playfair Display, IBM Plex Sans, IBM Plex Mono TYPOGRAPHY

Three typefaces used across the application. Playfair Display for headings (warm, editorial); IBM Plex Sans for body text (clean, neutral); IBM Plex Mono for numbers and technical readouts (monospace, alignment-friendly).

Brand differentiation; the Plex family is open source and looks professional at small sizes; Playfair gives editorial gravitas to headings without feeling corporate.

Server Backend

Google Cloud Run PROXY BACKEND HOST

A serverless container hosting service from Google Cloud. The Forge & Flow proxy backend runs as a container on Cloud Run, scales automatically with traffic, and shuts down to zero instances when idle.

Production keys must live somewhere with strong access control. Cloud Run provides container hosting plus secret management plus identity-based authentication without the operational overhead of managing virtual machines or Kubernetes. Cold-start latency is the trade-off (5-15 seconds on first request after idle); accepted at MVP, switched to `min-instances=1` at multi-operator launch (Q11).

Supabase (Postgres) SERVER DATABASE

Managed Postgres hosting. Forge & Flow rents Supabase to host the production database. During development, a local Supabase Docker container provides the same environment locally.

Postgres is the most-trusted relational database. Supabase eliminates the operational overhead of running it. Native row-level-security support (Phase 9). Built-in authentication that integrates cleanly. `db push` and `db diff` commands handle schema migrations.

Apache AGE GRAPH EXTENSION

A Postgres extension that adds graph database capabilities. Forge & Flow stores the methodology knowledge graph as nodes and edges in AGE, queryable via the Cypher query language, in the same Postgres instance as the relational tables.

The Advisor differentiator depends on graph traversal - "what shifts share these conditions?" is a natural graph question that maps awkwardly to relational joins. AGE keeps everything in Postgres rather than running a second separate database service like Neo4j. The Microsoft Azure production pattern of "AGE + pgvector + relational on one Postgres instance" was the precedent. Fallback architecture (Q12) supports vector-only retrieval if AGE cannot scale at production load.

pgvector VECTOR EXTENSION

A Postgres extension that adds a **vector** data type and similarity-search functions. Forge & Flow stores the 1024-dimensional Voyage embeddings of corpus chunks in pgvector and finds most-similar chunks via approximate nearest-neighbor (HNSW) indexes.

Industry standard. Hierarchical Navigable Small World (HNSW) indexes give sub-100-millisecond similarity search at production scale. Co-resident with AGE and the relational tables - one query planner, one connection pool, one backup strategy.

Artificial Intelligence Services

Anthropic Claude LANGUAGE MODEL

The large language model (LLM) the Advisor uses to generate answers. Two models are routed: **claude-haiku-4-5** for routine "quick" questions (default, ~12× cheaper) and **claude-sonnet-4-6** for nuanced questions that benefit from a smarter model.

*Claude has the strongest published track record on instruction-following, prompt-injection resistance, and "say no when you should" behavior. The model family is locked at launch but the **LLMProvider** abstraction (Promise #8) allows swapping in OpenAI or other providers later without rewriting consumer code.*

Voyage EMBEDDINGS & RERANK

Voyage builds two services Forge & Flow uses: **voyage-4-large** produces 1024-dimensional embeddings of text (used for similarity search over the corpus); **rerank-2.5** re-scores candidate chunks for relevance after vector search returns the top-K.

Voyage's embeddings consistently top the Massive Text Embedding Benchmark (MTEB). The 1024-dimension vectors balance accuracy and storage cost. The rerank model meaningfully sharpens retrieval quality at low cost. Same provider abstraction as the LLM means the company can be swapped out later without rewriting consumer code; embedding swaps trigger a corpus re-embed because vector dimensions vary across providers.

Model Context Protocol (MCP) Tool Layer AGENT TOOLS

An Anthropic-published standard for letting AI agents call tools (functions). Forge & Flow exposes safe, read-only operator data tools to the Advisor through an MCP layer that lives inside the proxy backend (Phase 11a.10).

Standardized agent tool-calling protocol means agents from any compatible model can use the same tools. Read-only at launch (Phase 11b); write-tools land in Phase 12 with explicit per-action operator consent and stricter bounds.

Authentication & Authorization (Phase 9)

Firebase Authentication USER AUTH (PHASE 9)

Google's hosted authentication service. Issues identity tokens that the proxy backend verifies to know which operator is making a request. Used for operator login and for the per-operator scope that drives row-level-security enforcement.

Mature, free at low scale, integrates cleanly with Cloud Run and Supabase. Eliminates the security-critical work of building authentication from scratch.

Development & Operations

Git & GitHub VERSION CONTROL

Source control. Forge & Flow's repository lives on GitHub. Commits at phase close (not slice close); push is automatic on commit via a post-commit hook.

Industry standard.

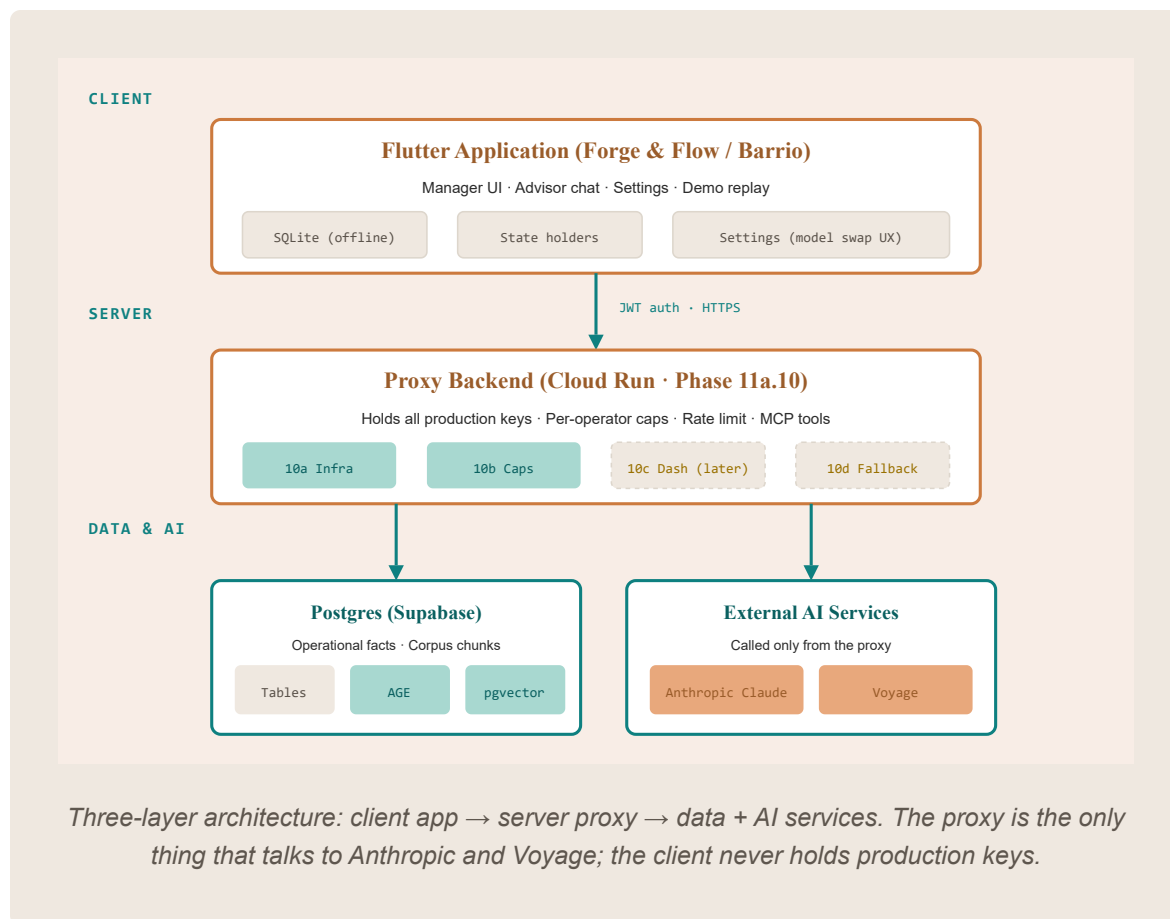
graphify CODE & DOC GRAPH

A tool that builds a navigable knowledge graph of the codebase and documentation. Used for orientation when exploring unfamiliar parts of the codebase. Auto-rebuilds via a post-commit hook when code changes; flags the doc graph for refresh when documents change.

Token efficiency. Codex and Claude can answer "where does X live and what depends on it?" against the graph before reading raw files, saving significant context budget on every session.

Architecture Overview - How the Pieces Fit Together

At the highest level, Forge & Flow has three layers: the application running on the operator's device, the proxy backend hosted on Cloud Run, and the data + artificial-intelligence services behind the proxy.



Why this shape

Restaurants cannot manage API keys. A "bring your own key" pattern would require operators to register with Anthropic, manage billing, rotate keys, and worry about rate limits. None of that fits the user - restaurants want a subscription that just works. The proxy backend exists precisely to absorb all of that complexity. Forge & Flow holds the keys, eats the cost, and bills operators a flat subscription tied to their pricing tier (Hard Promise #7).

Per-operator caps and rate limits live where keys live. If the cap and rate-limit logic ran in the client application, a misbehaving client could circumvent them. By placing both in the proxy, Forge & Flow guarantees that no individual operator's runaway usage can bankrupt the company.

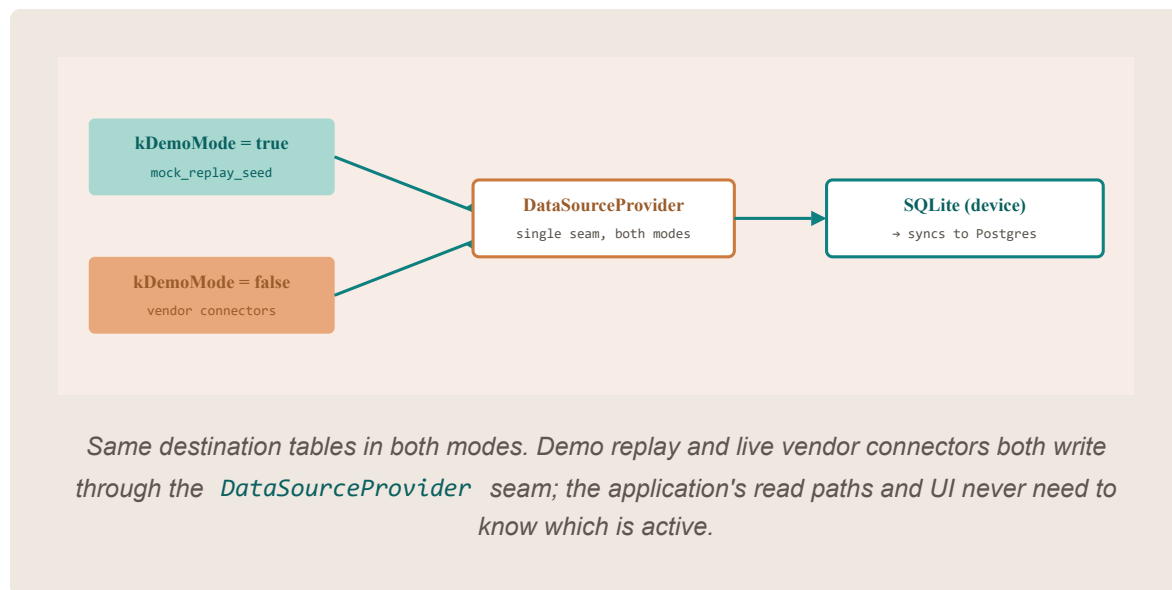
The graph and the vectors live in the same Postgres instance. Apache AGE for graph traversals (the differentiator) and pgvector for similarity search (the retrieval substrate) coexist with relational tables in one Postgres instance. One backup, one connection pool, one query planner. The Microsoft Azure pattern is the precedent. The fallback architecture (Q12) supports a vector-only retrieval mode if AGE cannot scale.

The application is offline-first. Restaurant networks fail. Service must continue. SQLite on the device persists operational data; synchronization to Postgres happens when the network is available.

Data Flow - How Information Moves Through the System

Operational data flow (writes)

Operational data - shifts, sales, schedules, variance - enters the system from one of two sources, depending on whether the application is in demo mode or live mode. Either way, the destination tables, read paths, and user interface are identical (Hard Promise #2).



The `DataSourceProvider` abstraction (introduced in Phase 7.57.3a, locked through Phase 7.57.3c) is the seam that makes Phase 8 a "pure transport swap" rather than a redesign. Vendor connectors implementing `DataSourceProvider` simply replace the demo replay implementation in production without changing anything downstream.

Advisor question flow (reads)

When an operator asks the Advisor a question, the flow is:

1. The Flutter application sends the question over HTTPS to the proxy backend, attaching the operator's authenticated identity (a JSON Web Token from Firebase Authentication).

2. The proxy backend verifies the identity and checks the operator's monthly cost cap and rate limit. If either is exceeded, it refuses the call.
3. The proxy retrieves operator-relevant context from the Postgres database - methodology corpus chunks (via pgvector similarity search, then Voyage rerank), graph-traversed related concepts (via Apache AGE Cypher queries), and operator's actual data (via MCP tool calls scoped by `(operator_id, location_id)`).
4. The proxy assembles a prompt with system instructions (defining what the Advisor is and is not), the retrieved context wrapped in clearly delimited blocks, and the operator's question.
5. The proxy calls Anthropic's Claude API. Default model is `claude-haiku-4-5` for the "quick" tier; `claude-sonnet-4-6` for "nuanced" tier.
6. Claude generates an answer. Output sanitization strips or rewrites any markdown links or images in the response that do not point to allowlisted domains (defense against the EchoLeak-class data exfiltration attacks).
7. The sanitized answer returns to the Flutter application and renders in the chat surface with provenance - which corpus chunks were cited, which tools were called.

SINGLE-SHOT AT LAUNCH

This entire flow is **one** Claude call per question - single-shot Retrieval-Augmented Generation. Phase 12 post-launch will add agentic flows where Claude may call tools repeatedly, with hard bounds on tool-call count, token spend, and wall-clock time, plus mandatory human approval before any irreversible action.

Corpus update flow

When Vanessa drops a new methodology Markdown file into the Settings page (the `11a.12` slice that ships the corpus admin user experience), the system:

1. Reads the Markdown.
2. Splits it into chunks per the established chunking profile.
3. Computes a content hash of each chunk. Chunks whose hash already exists in the corpus stay unchanged; only chunks with new hashes are processed onward.
4. Sends new chunks to Voyage for embedding. The 1024-dimensional vector and provider/model metadata (provider id, model id, dimension) are stored alongside the chunk.
5. Writes new chunks (text + hash + embedding + metadata) into the Postgres corpus table.
6. Marks any old chunks whose content was replaced as inactive in the search index - but does not delete them. Old advisor recommendations that cited those chunks remain replayable forever.

Logic Layers - Where Code Lives and Why

The Flutter application's source tree is split into directories with strict rules about what each one is allowed to contain. The split makes the codebase legible, prevents drift, and keeps tests stable.

lib/screens/ · lib/widgets/

User interface. Widgets do not own source-truth or service-period bucketing. Display only.

lib/state/

State holders, refresh coordinators, runtime invalidation buses. Owns "what the UI is currently showing."

lib/services/

Runtime orchestration. Reads from infrastructure, calls domain formulas, returns shaped data to state.

lib/domain/services/

Pure formulas. No input/output side effects.

LaborModel, target-snapshot builders, provider interfaces.

lib/infrastructure/persistence/sqlite/ · /postgres/ Database helpers. Raw

package:postgres imports forbidden anywhere else (CI lint enforces).

*The service-layer split, codified in **CLAUDE.md** and enforced by Phase 7.57.0 governance rules.*

Frozen legacy: lib/data/

Before Phase 7.57, all of the above lived flat in **lib/data/**. That folder is now **frozen**: delete-only, never add. Files migrated out of **lib/data/** across Phase 7.57.2 sub-slices into the appropriate target directory above. The remaining files in **lib/data/** are the **kDemoMode** writer and a few canonical runtime defaults; everything else has moved.

Repository pattern (database-side defense)

Application code in `lib/services/` and `lib/domain/` never queries operator-scoped Postgres tables directly. Every read or write goes through an `OperatorScopedRepository<T>` that injects `(operator_id, location_id)` from the active session context. Postgres row-level-security is the safety net beneath; the repository is the primary defense. This is the two-layer pattern from Q5.

```
// Forbidden:
final shifts = await pg.query('SELECT * FROM shifts WHERE date = ?', [yesterday])

// Required:
final shifts = await operatorScopedShifts.find(date: yesterday);
//   ↳ compiles to "WHERE operator_id = ? AND location_id = ? AND date = ?"
```

Provider abstractions (artificial-intelligence side)

Every external AI service is wrapped in a capability-named interface in

`lib/domain/services/` :

- `LLMProvider` - chat completion (Anthropic, OpenAI, Gemini behind one interface)
- `EmbeddingProvider` - text-to-vector (Voyage today, others swappable)
- `RerankProvider` - candidate reranking (Voyage today)
- `DataSourceProvider` - operational data writers (demo replay today, vendor connectors after Phase 8)

The interfaces are general-purpose by design (Hard Promise #8). The Advisor in Phase 11b is one consumer; workflow automation in Phase 12 will be another using the same plumbing.

Schema Decisions - The Database Lock-Ins

Four decisions from the long-term-risk session on 2026-04-25 are **schema-level** - they shape how data is laid out in Postgres. Each is "irreversible after data exists." Once thousands of facts have been written, retrofitting any of these requires a painful data migration. They were locked early on purpose.

Q3: Multi-location schema

Every operator-scoped fact carries (operator_id, location_id)

DECIDED 2026-04-25 • LOCKED IN PHASE 7.57.0 GOVERNANCE RULES

Options considered. (A) `operator_id` only - multi-location operators run multiple operator accounts. (B) `(operator_id, location_id)` from day one. (C) `operator_id` + nullable `location_id`.

Chose B.

Single-location operators run with a default `location_id`. Corpus and methodology stay `operator_id`-scoped (one playbook covers all locations). Phase 9 row-level-security policy becomes `(operator_id, location_id)`, not `operator_id` alone. Retrofit is not an option once thousands of facts exist; the cost of carrying the column today is trivial; the cost of adding it later is brutal.

Q4: Time-storage convention

TIMESTAMPTZ + denormalized business_date computed at write

DECIDED 2026-04-25 • TIME-BOUNDARY CONTRACT RULE 11

Options considered. (A) Coordinated Universal Time only, compute business date at read. (B) UTC instants plus a denormalized `business_date` column computed at write. (C) Time-zone-naive timestamps storing local wall-clock time.

Chose B.

Source-truth instants are stored as `TIMESTAMPTZ` (UTC). A `business_date` `DATE` column is computed once at write using `Location.timezone` (an Internet-Assigned-Numbers-Authority string per location) plus `business_day_rollover_hour` per location, then never recomputed. `TIMESTAMP WITHOUT TIME ZONE` is banned in operator-scoped tables. Daylight-saving-time corruption on time-zone-naive timestamps is silent and unrecoverable; option C was a non-starter despite being the easiest. Indexes on `business_date` are natural for "shifts on a given day" queries; option A would require every query to join against `Locations.timezone` and recompute.

Q5: Repository pattern + RLS as backup

App code never queries operator-scoped Postgres directly

DECIDED 2026-04-25 • LOCKED IN PHASE 7.57.0 GOVERNANCE RULES

Options considered. (A) Postgres row-level-security alone (rely on the database to enforce). (B) Repository wrapper enforces, with row-level-security as backup. (C) Per-operator views/schemas (physical isolation).

Chose B.

Row-level-security alone has shipped data leaks at multiple production multi-tenant systems - a missed policy on a new table or a `BYPASSRLS` grant slipped through review and the leak is silent. The repository pattern makes the unsafe path syntactically impossible in application code, enforced by a continuous-integration lint rule that forbids raw `package:postgres` imports outside `Lib/infrastructure/persistence/postgres/`. Row-level-security stays enabled underneath as the safety net. Two-layer defense - repository wrapper as primary, row-level-security as backup. Per-schema isolation (option C) was over-engineering at scale (1000 operators = 1000 schemas).

Q6: Corpus content-addressing

Chunk identifiers are content hashes, not mutable identifiers

DECIDED 2026-04-25 • LOCKED FUTURE TWEAK 11A.11

Options considered. (A) Mutable identifiers - chunk content can change in place, identifier stays. (B) Content-hash identifiers - change content, get a new identifier. (C) Versioned identifiers - both stored.

Chose B.

Methodology will evolve. Old advisor recommendations cited specific chunks; if those chunks are mutated in place, the advisor's history is silently rewritten. Content-hashed identifiers preserve the audit trail forever. Old chunks stay in the corpus marked inactive in the search index. Re-embedding runs only on chunks with new hashes - efficient and incremental. Same pattern Git uses for the same reason.

Artificial-Intelligence Infrastructure - The Heart of the Product

The AI infrastructure is what makes Forge & Flow more than a dashboard. It is also the component with the most distinct design decisions, the highest risk surface, and the most operational care.

The provider abstraction layer

Every external AI service sits behind a capability-named interface (Hard Promise #8). The interfaces live in `lib/domain/services/` :

Interface	Implementations today	Capability flags
LLMProvider	ClaudeAnswerProvider (Anthropic)	promptCaching , toolUse , vision
EmbeddingProvider	VoyageEmbeddingProvider	(provider id, model id, dimension)
RerankProvider	VoyageRerankProvider	-
DataSourceProvider	MockReplayDataSourceProvider	-

Adding a second LLM provider - for example OpenAI's `gpt-4o-mini` for fallback - means writing one new `OpenAIAnswerProvider` implementation. Consumer code (the Advisor, future workflow automation) depends only on the interface. This is what makes Hard Promise #8 ("AI infrastructure is general-purpose") concrete.

The Settings page exposes the routing. A development-and-administrator-only Settings section (`settings_advisor_model_section.dart`) lets the founder swap which provider/model handles which tier without redeploying. Visible only when running in development mode or when the operator role is administrator.

The proxy backend (Phase 11a.10)

The proxy backend is the launch-blocking foundation of the entire commercial story (Hard Promise #7). It splits into four sub-slices:

Slice	What it does	Status
11a.10a	Cloud Run service · key vault · JWT auth · operator scoping	LAUNCH
11a.10b	Per-operator token budget · rate limit · monthly cost cap with refusal · counter table	LAUNCH
11a.10c	Admin dashboards · daily/monthly aggregates · alert thresholds	POST-100-LOC
11a.10d	Provider fallback chains (Anthropic → OpenAI → Gemini)	CAPABILITY OFF

Cost containment

Because Forge & Flow eats every token (Hard Promise #7), per-operator cost predictability is a launch gate. Three levers stack:

- **Tier routing.** `quick` tier defaults to `claude-haiku-4-5` (about 12× cheaper than Sonnet); `nuanced` tier escalates to `claude-sonnet-4-6` only when the question's complexity earns it.
- **Anthropic prompt caching.** Long system prompts and methodology context are cached; repeated calls pay roughly one-tenth the cost on the cached portion. Exposed through the generic `Capability.promptCaching` flag.
- **Per-operator monthly cap.** Hard refusal when the operator hits their tier's monthly token budget. Configurable per pricing tier.

Cost reality at Haiku-first tiered routing with prompt caching

Operator profile	Cost per month
Heavy operator (~100 questions/month, mostly Haiku-routed)	\$0.10–\$0.30
Light operator (~20 questions/month)	\$0.02–\$0.06
Embedding (one-time + on corpus update at 233 chunks)	~\$0.05 total
Per-operator monthly cap (configurable per tier)	\$5.00 default

A \$5 cap leaves room for healthy margin on a typical SaaS subscription. The real cost risk is uncapped agentic loops in Phase 12 - which is why agent runaway bounds (Q10) and Haiku-default routing must be in place before any write-tool ships.

Prompt-injection defense

Prompt injection is the attack class where an attacker (or careless input) sneaks instructions into untrusted content that the language model then follows. Operator-typed shift notes, schedule comments, and similar free-text fields are the realistic attack surface for Forge & Flow.

IMPORTANT FRAMING

The 2025 industry consensus (Anthropic, OWASP, Microsoft, Simon Willison) is that **no prompt-level defense alone is reliable**. The architecture is the defense; the prompt-stack is hygiene on top. Forge & Flow's row-level-security plus repository pattern plus read-only Advisor at launch plus single-shot Retrieval-Augmented Generation plus founder-controlled corpus is the actual safety. The hardening below is two-layer protection above that foundation.

Launch defense stack (research-updated 2026-04-25)

1. **Base64-encode operator free-text fields when injecting them into context.** Microsoft Spotighting "encoding" variant - published attack-success rate roughly 2% versus 50% for naive concatenation. The system prompt instructs Claude to decode mentally and treat the result as data, never as instructions.
2. **System-prompt scoping.** Explicit instruction: "you only answer questions about restaurant operations using the provided context; decline anything off-topic." Open Web Application Security Project's number-one mitigation; free.

3. **Output sanitization for markdown links and images.** Strip or rewrite any link or image in the model's response unless it points to an allowlisted Forge & Flow-internal domain. This is the defense that would have prevented the Slack AI exfiltration (August 2024) and Microsoft EchoLeak (June 2025) - both real production data leaks. Cheap; high-impact.
4. **Length caps** on operator free-text fields when injected into context (e.g., 500 characters on shift notes - separate from storage limits). Large injected blobs are an attack vector.
5. **Anthropic role separation.** Used for structural clarity; published 480-test study showed negligible security impact alone, so it is hygiene, not a counted defense layer.

Multi-operator launch additions

- Input-side classifier on operator queries (Meta's Llama Prompt Guard 2 - free, fast, multilingual)
- Provenance markers in every context block (`<source operator="..." trust="...">`)
- Retrieval-scope audit - filter by `operator_id` before vector similarity, not after
- Adversarial test suite in continuous integration

Phase 12 (write-tools and agent loops)

- Plan-Then-Execute or Action-Selector pattern (Simon Willison design patterns paper, June 2025) - tool calls decided before the model sees retrieved or operator-supplied content
- Two-Language-Model separation (CaMeL-style) for any write path that touches operator-supplied content
- **Human-in-the-loop mandatory for irreversible actions** (schedule changes, communications, anything financial)
- Tool-output classifier over every tool result before it re-enters agent context
- Egress allowlist - closed list of where automated workflows can reach
- Per-operator audit log of every tool call

Agent runaway bounds (Q10)

At launch, the Advisor is single-shot - one Claude call per question. The bounds are simple: per-request `max_tokens` and per-request HyperText-Transfer-Protocol wallclock timeout (default 30 seconds), plus the per-operator monthly cost cap. These catch single-request runaway with native Application-Programming-Interface features.

For Phase 12 (when write-tools land), four additional bounds apply per task:

- Maximum tool-call count (default 20)
- Maximum token spend (default \$1.00 / 100,000 tokens)
- Maximum wall-clock time (default 60 seconds)

- Maximum parallel tasks per operator (default 3)

Failure mode is **graceful summary**, not hard cut-off. When a limit hits roughly 80%, the agent runtime injects "summarize what you have found so far and conclude." The operator sees a partial result with a "task was cut short" badge, never an error wall.

Security and Privacy

Per-operator isolation

Two layers of defense, one architectural:

- **Repository pattern (primary).** Application code can never write a query that omits the `(operator_id, location_id)` filter. The repository wrapper compiles every query with those columns injected from the active session context. Continuous-integration lint enforces no raw `package:postgres` imports outside the persistence layer.
- **Row-Level Security (backup).** Postgres row-level-security policies enforce `(operator_id, location_id)` at the database engine. If application code somehow constructs an unsafe query, the database refuses it.
- **Architectural defense.** Read-only at launch. No write-tools. Single-shot Retrieval-Augmented Generation. Founder-controlled corpus with no operator-supplied global content. The blast radius if injection somehow succeeds is bounded to "the model says something wrong about a single operator's data."

Key management

Production Anthropic and Voyage keys live exclusively in the Cloud Run environment plus Google Cloud's Key Management Service. Flutter release builds and App Store / Play Store builds receive **no** production keys. The development `--dart-define` path is only for local development.

The `.env.local` file is git-ignored. The `.env.local.example` file is git-ignored too - after a real key was once accidentally placed in it, the gitignore was extended to cover example files explicitly.

Liability posture

The Advisor is recommendation-only (Hard Promise #6). Operator decisions and outcomes are the operator's sole responsibility. Phase 9.8 codifies this in operator-facing terms-and-conditions: Forge & Flow provides advisory information; the operator decides whether to act and is solely responsible for outcomes. Errors-and-omissions insurance evaluation also lives in Phase 9.8.

Audit trail

A light advisor audit log is decided but not implemented at launch (parked for Phase 11b.2 polish). The schema is locked: question, answer, timestamp, `operator_id`, cost. Heavier audit (corpus version snapshot, retrieved chunk identifiers, replay tool) is a post-launch decision if a dispute ever surfaces. Recommendation-only posture is the legal shield; the audit log is operational hygiene.

The Build Phase Plan - Sequential

Cadence

The build is sequential, not parallel. One phase finishes before the next opens. The locked order is:

[7.57] → 11a → 9.8 → 9 → 10a → 10.5 → 9.5 →
[7.58] → 11b → 9.75 → 11b.2 → 10b →
[7.61] → 8 → 8R

Phase descriptions:

Phase	Status	What it does
7.57	ACTIVE	Structural stabilization. Service-layer split, provider abstractions, Apache AGE projection. Slices 7.57.0 through 7.57.4.
11a	PAUSED 11A.7	Advisor infrastructure: corpus pipeline, vector search, rerank, proxy backend (10a/10b launch-blocking, 10c/10d deferred). Resumes after 7.57.4.
9.8	QUEUED	Compliance and legal: terms-and-conditions, output-sanitization allowlist, pricing tier model.
9	QUEUED	Authentication and per-operator identity. Row-level-security enforcement turns on here.
10a	QUEUED	Multi-device shared state.
10.5	QUEUED	Daypart Shift view alongside the existing whole-day view.
9.5	QUEUED	El Podio learning identity (per-staff-member identity for Barrio).
7.58	QUEUED	Pre-11b behavioral alignment lane. First slice that decides on existing application logic. Five sub-slices.
11b	QUEUED	Advisor user-interface - the chat surface inside Forge & Flow and Barrio.
9.75	QUEUED	Staff daily companion (Barrio shell).
11b.2	QUEUED	Advisor polish (light audit log conversation lives here).
10b	QUEUED	Full offline synchronization.
7.61	QUEUED	Pre-Phase-8 freshness audit. Four sub-slices.
8	QUEUED	Live point-of-sale and labor connector - the pure transport swap.
8R	QUEUED	Live reservation connector.
12	POST - LAUNCH	Workflow automation lane (write-tools, agent loops, two-language-model separation, human approval).

Why sequential and not parallel

The earlier 2026-04-22 framing had Phase 11a "running in parallel with Phase 8 / 8R / 9." That framing was superseded on 2026-04-25 by the sequential lock. The reasons:

- **Time before launch.** Sequential is slower in calendar time but produces a more stable system. The team has time before launch and is choosing stability.
- **Decision dependencies.** Many decisions in later phases want signal from earlier phases. Phase 9.8 pricing tiers depend on real cost data from Phase 11a. Phase 11b user-interface depends on Phase 9 authentication. Sequential makes those dependencies legible.
- **One-person-at-a-time work.** The team is small. Parallel lanes create cognitive switching cost without a corresponding throughput gain.

Decision Log - Q1 through Q12 with Rationale

The 2026-04-25 long-term-risk session walked through twelve decisions in priority order - Tier 1 schema lock-ins first (irreversible after data exists), then Tier 2 architectural shape (compounding but addressable), then Tier 3 operational. Each decision is locked with rationale.

Q1 - Advisor liability posture

TIER • POSTURE • 2026-04-25

Should the Advisor take action, or recommend?

Chose: Recommendation-only.

F&F provides advisory information; operator decides; F&F not liable. Codified in Phase 9.8 terms-and-conditions. Workflow automation in Phase 12 extends with explicit per-action consent.

Q2 - Audit log depth

TIER • LIABILITY • 2026-04-25 • NO IMPLEMENTATION

(A) light audit / (B) full audit with replay metadata / (C) tiered / (D) full + replay tool.

Chose: A (light), no implementation now, parked for 11b.2 conversation.

Recommendation-only posture is the legal shield; audit log is operational hygiene. Light schema (question, answer, timestamp, operator_id, cost) is enough until a real dispute surfaces.

Q3 - Multi-location schema

TIER 1 • SCHEMA • 2026-04-25

(A) operator_id only / (B) (operator_id, location_id) day one / (C) nullable location_id.

Chose: B from day one.

Retrofit on thousands of facts is brutal; column cost today is trivial; org-level operators with multiple locations are the highest-margin segment.

Q4 - Time-zone storage

TIER 1 • SCHEMA • 2026-04-25

(A) UTC only / (B) TIMESTAMPTZ + business_date / (C) timezone-naive timestamps.

Chose: B.

DST corruption on timezone-naive timestamps is silent and unrecoverable. POS / reservation industry-standard pattern. Indexes on business_date are natural for daily queries.

Q5 - Row-Level Security pattern

TIER 1 • DEFENSE • 2026-04-25

(A) Postgres RLS only / (B) Repository wrapper + RLS as backup / (C) Per-schema isolation.

Chose: B.

Production multi-tenant systems converge on B. RLS catches the bug; the wrapper prevents the bug. Per-schema isolation does not scale to thousands of operators.

Q6 - Corpus content addressing

TIER 1 • CORPUS • 2026-04-25

(A) Mutable IDs / (B) Content-hash IDs / (C) Versioned IDs.

Chose: B + new 11a.x slice for markdown corpus admin UX in Settings (Vanessa-self-serve).

Same pattern Git uses. Old advisor citations stay accurate forever. Re-embedding runs only on new hashes.

Q7 - Provider fallback when Anthropic is down

TIER 2 • RESILIENCE • 2026-04-25

(A) Hard fail at launch / (B) Cached prior answer / (C) Provider fallback chain / (D) Multi-provider primary.

Chose: A at launch. B as locked future tweak. C as locked future capability inside 11a.10d (off by default).

Anthropic uptime ~99.5% - outages rare and short. Fallback chain has subtle answer-quality drift between providers; turn on only when production data shows it is needed.

Q8 - Streaming responses

TIER 2 • UX • 2026-04-25

(A) Wait then show / (B) Stream tokens / (C) Both methods on interface.

Chose: A at launch. Streaming added later as additive `completeStream()` - non-breaking.

Pre-launch validation is about answer substance, not UX feel. Streaming-on-top-of-proxy adds 1-2 weeks of Server-Sent-Events work that does not change correctness.

Q9 - Prompt-injection defense

TIER 2 • SECURITY • 2026-04-25 • RESEARCH-UPDATED

(A) Trust the LLM / (B) XML delimiter wrapping / (C) Anthropic role separation / (D) Output filtering / (E) Defense in depth.

Chose: research-updated stack - base64 encoding (Microsoft Spotlighting "encoding" variant) + system-prompt scoping + output sanitization for markdown links/images + length caps. Multi-operator and Phase 12 layered defenses also documented.

Initial XML-delimiter framing was the weakest of three Spotlighting variants. Base64 reduces attack success from ~50% to ~2%. The single most impactful add was output sanitization, which would have prevented Slack AI (Aug 2024) and Microsoft EchoLeak (June 2025) - both real production exfil incidents.

Q10 - Agent runaway bounds

TIER 2 • COST • 2026-04-25

Per-request limits at launch; per-task limits for Phase 12.

Chose: per-request max_tokens + wallclock at launch (11a.10b extension); per-task tool-call/token/wallclock/parallel caps in Phase 12 with graceful summary fallback.

Single-shot RAG at launch only needs per-request bounds (catches single-request runaway). Phase 12 agent loops need per-task accounting; graceful summary at 80% budget beats hard cut-off for UX.

Q11 - Cloud Run cold-start strategy

TIER 3 • OPERATIONS • 2026-04-25

(A) min-instances=0 / (B) min-instances=1 / (C) Warmup pings.

Chose: A at MVP (\$0/month, accept ~12s cold start). B at multi-operator launch (\$15-30/month, eliminate cold start).

Single-operator beta tolerates infrequent cold-start; cost not justified pre-revenue. Multi-operator usage spread across day means proxy never naturally goes cold; min-instances=1 just guarantees it.

Q12 - Apache AGE graph fallback

TIER 3 • RESILIENCE • 2026-04-25

(A) No fallback / (B) Documented fallback to vector-only / (C) Switch to Neo4j now.

Chose: B. Retrieval layer in 11a.8/11a.9 ships with a mode flag; AGE default; vector-only as dormant insurance.

F&F should not catastrophically fail on a differentiator's bad day. Adding the mode flag costs ~half a day; the Neo4j migration if ever triggered is a 6-9 week swap project (the abstraction makes it bounded). AGE benchmark gate at p95 > 500ms decides whether vector-only flip is the launch posture.

Future Flags - Items Parked for Later Decision

Decisions that don't belong in 7.57 / 7.58 / 7.61 but need a decision at the right moment. Codex prompts for them when their phase opens.

PHASE 9.8 - COMPLIANCE & LEGAL

- **Advisor terms-and-conditions and liability disclaimer codification.** Operator-facing T&Cs explicitly stating F&F provides advisory information; operator decisions and outcomes are the operator's sole responsibility. Plus errors-and-omissions insurance evaluation for AI advisory output. Posture is locked (recommendation-only, Hard Promise #6); this flag tracks the legal/insurance work.
- **Production observability - errors only.** General application error tracking. Cheap path: Sentry free tier.
- **F&F pricing tier model.** What subscription tiers exist; what cost cap each tier gets; which tier gets which model (Haiku-only versus Sonnet-on-demand). Cost reality numbers documented (heavy operator \$0.10–\$0.30/month at Haiku-first, light \$0.02–\$0.06/month, \$5/operator/month cap leaves margin).
- **Output sanitization allowlist.** What URL domains are permitted in advisor responses. At minimum: an F&F-internal allowlist of allowed link/image hosts.

PHASE 11A.10C - COST OBSERVABILITY DASHBOARDS

Per-operator dashboards, daily/monthly aggregates, alert thresholds. **Deferred 2026-04-25: too much overhead before 100 locations.** The counter table from 11a.10b is enough for cap enforcement until volume justifies the dashboard build.

PHASE 11A.10D - PROVIDER FALLBACK STRATEGY

What's the actual chain order - Anthropic → OpenAI gpt-4o-mini, or Anthropic → Google Gemini Flash, or three-deep? Cost-vs-reliability trade-off. Capability is locked; strategy choice is the flag.

PHASE 11A.11 - PRODUCTION DATABASE MIGRATION STORY

Local Supabase container → cloud Supabase. Built-in via Supabase `db push` , `db diff` , point-in-time recovery.

PHASE 11B.2 - ADVISOR AUDIT LOG (LIGHT)

Question, answer, timestamp, `operator_id` , cost. **No implementation now**; parked for 11b.2 conversation. Pairs with the Phase 9.8 T&Cs flag - recommendation posture is the legal shield, audit log is operational hygiene. Heavier schema (corpus version, retrieved chunks, prompt hash, replay tool) is post-launch only if a dispute surfaces.

NEAR PHASE 10B CLOSE - LAUNCH READINESS CHECKLIST

Day-1 readiness pass. Markdown document.

POST-LAUNCH (PHASE 12 CANDIDATE) - WORKFLOW AUTOMATION LANE

Builds on the same provider abstraction + MCP tool layer. Extends read-only tools to write-tools (schedule edits, notifications, plan adjustments). Per-workflow cost caps and agentic loop limits required before any write-tool ships. Demand signal: operator asks for "AI runs operations," or 10+ operators asking for the same workflow.

Quick Reference Card

The eight hard promises in one line each

1. Phase 8 is a pure transport swap.
2. Demo mode persists post-launch.
3. No application-logic changes before Phase 7.58.
4. Per-operator isolation, multi-location-ready from day one.
5. Apache AGE graph projection ships before Phase 11b.
6. Advisor speaks in recommendations, not commands.
7. F&F holds all provider keys server-side.
8. AI infrastructure is general-purpose.

Schema rules in one line each

- Every operator-scoped fact carries `(operator_id, location_id)`.
- `TIMESTAMPZ` + write-once `business_date`; never `TIMESTAMP WITHOUT TIME ZONE`.
- App code never queries operator-scoped Postgres directly; repository wrapper + RLS backup.
- Corpus chunks are content-hashed; old chunks stay inactive but never delete.

Service-layer split in one line each

- `lib/data/` - frozen legacy. Delete-only.
- `lib/services/` - runtime orchestration.
- `lib/domain/services/` - pure formulas, no I/O.
- `lib/state/` - state holders.
- `lib/dev/` - demo and dev-only material.
- `lib/infrastructure/persistence/` - DB helpers.

The build order

```
[7.57] → 11a → 9.8 → 9 → 10a → 10.5 → 9.5 →
[7.58] → 11b → 9.75 → 11b.2 → 10b →
[7.61] → 8 → 8R
```

Authority order (when sources conflict, later loses to earlier)

1. The active prompt.
2. `docs/contracts/**`
3. `PROJECT_TRACKER.md` , `docs/DATA_ALIGNMENT_TRACKER.md`
4. The active phase doc named in the prompt.
5. `CLAUDE.md`

Cost reality at Haiku-first tiered routing with prompt caching

- Heavy operator (~100 questions/month): \$0.10–\$0.30/month.
- Light operator (~20 questions/month): \$0.02–\$0.06/month.
- \$5/operator/month cap leaves healthy margin on a typical SaaS subscription.
- Real cost risk is uncapped agentic loops in Phase 12.

CLOSING THOUGHT

The architecture is the safety net. The prompt-stack is hygiene on top of it. Row-level-security plus repository pattern plus read-only Advisor at launch plus single-shot Retrieval-Augmented Generation plus founder-controlled corpus is the actual defense. Everything else is two-layer protection above that foundation.